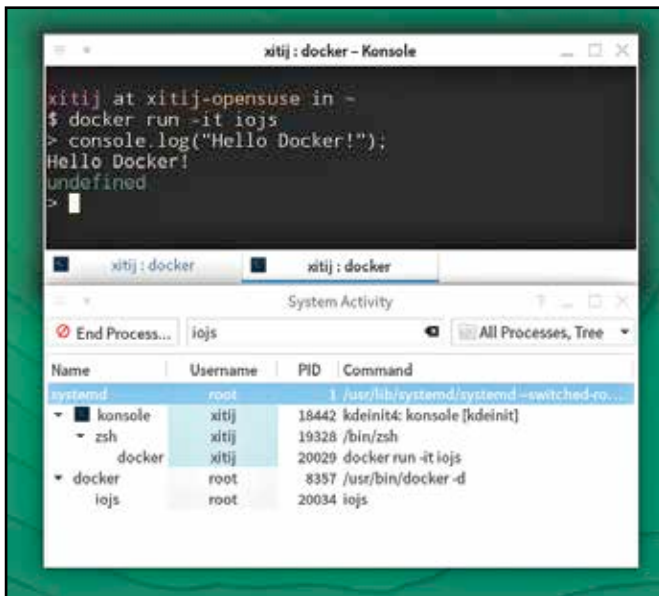




As you may notice, the running iojs process is directly visible on the host system.

Just change 'ubuntu' to 'centos' or 'fedora' and you are good to go! NOTE: Wherever we use the docker command, you may need to prefix sudo depending on how it is configured on your OS.

This works because in addition to all the magic Docker does on your own system, it also comes configured with an online repository of images of different operating systems. It can automatically pull any OS or application that someone has uploaded to their registry, and install it and run it on your system.

Once it has been downloaded and installed, anytime you run the same command in the future, it will not need to download and install Ubuntu again, it will simply reuse the existing install. So future invocations will run instantly, and show an output in under a second. It's almost as if it's running on your own system directly! Well, that's the thing, it is.

On the shoulders of giants

If you've used virtual machine software like VirtualBox and VMWare, this might seem a little familiar, while at the same time a little fishy. Virtual machine software boot an entire operating system with its own kernel in order to give you access to an isolated environment in which you can run software. How does Docker do it instantly?

The answer is that instead of isolating an entire machine, Docker uses features of Linux to isolate just the bits of software that need to be isolated. It puts constraints only on this environment.

Two of the most important features that enable this are cgroups (or control groups) and namespaces. There is a LOT to both these technologies, but here they are in brief:

- **Namespaces:** Namespaces allow multiple applications running on a system to think they are running in different environments. For instance two applications running in different namespaces will not 'see' each other if they ask the system for a list of running applications. Namespaces can also give people a different view of the file-system, and network etc.
- **cgroups:** These are a Linux feature that allow one to place restrictions on how many resources they consume. For instance you

could launch an application while limiting the RAM it uses to 100MB, or limiting its usage of the CPU.

As you can see, just by combining the above two, you can get features similar to that of a virtual machine, i.e. applications running in a different isolated environment, with a controlled amount of resources.

In addition to the above two, Docker also makes extensive use technologies such as btrfs / aufs, systemd and libvirt.

Basic Docker Usage

The simplest way to get started with Docker is to use the run command as we had demonstrated before. However, there is a lot more you can do with Docker than just that.

While the run command automatically downloads an image when you run it, you can also just ask Docker to download an image using docker pull.

For instance if you want to check out io.js, just run:

This will download an OS that has io.js pre-installed. After this you can use the docker run command to run applications in this image. At this point if you wanted to try io.js you should run.

The '-i' and '-t' here tell Docker to run in interactive mode, so you can enter commands rather than just see output. Notice, how unlike before we have not really told it which command to run inside the io.js image. This is because the way the io.js image is built, simply running it will drop you into the io.js interpreter. You can still specify commands in which case they will be run instead of the default command.

Docker Mechanics

Lot's of interesting stuff happens behind the scenes every time you use docker run. Here is what is going on behind the scenes when you do docker run. Docker first creates an entirely new, fresh environment that Docker calls a container built on top of the 'iojs' image it downloaded. In this container it then runs whatever command you specify, thus isolating any changes those commands make to the container, and leaving the image unaltered.

What this means is, every time you run a command as docker run ubuntu <command> that command runs in its own container! It runs as if on a fresh Ubuntu install. Any changes made by a previous run command are not visible to the next run.

But what if you want to continue where you left off. Well, the container with all your changes is still there. While docker run starts



This little Firefox is running in a Docker container.